

7.9 User Actions

7.9.1 File Actions

7.9.1.1 File.Close

Closes a document (see *Source Viewer* on page 155).

Prototype

```
int File.Close(const char* sFilePathOrName);
```

Argument	Meaning
sFilePathOr- Name	File path (or name) of a source file (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

Main Menu → Window → Close Document (Ctrl+F4)

7.9.1.2 File.CloseAll

Closes all open documents (see *File Menu* on page 44).

Prototype

```
int File.CloseAll();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Window → Close All Documents (Ctrl+Alt+F4)

7.9.1.3 File.CloseAllButThis

Closes all but the active document (see *Source Viewer* on page 155).

Prototype

```
int File.CloseAllButThis();
```

Return Value

-1: error
0: success

GUI Access

Document Tab → Context Menu → Close All But This (Ctrl+Shift+F4)

7.9.1.4 File.CloseAllUnedited

Closes all unedited documents (see *Source Viewer* on page 155).

Prototype

```
int File.CloseAllUnedited();
```

Return Value

-1: error
0: success

GUI Access

Document Tab → Context Menu → Close All Unedited Documents

7.9.1.5 File.Exit

Closes the application (see *File Menu* on page 44).

Prototype

```
int File.Exit();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Exit (Alt+F4)

7.9.1.6 File.Find

Searches a text pattern in source code documents (see *Find In Files Dialog* on page 69).

Prototype

```
int File.Find(const char* sFindWhat);
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Find → Find In Files (Ctrl+Shift+F)

7.9.1.7 File.Load

Downloads a program or data file to target memory. This command essentially performs the same operation as File.Open, but it does not reset the target prior to download and does not perform the initial program operation (see *Download Behavior Comparison* on page 197). When an ELF or compatible program file is specified, its debug symbols replace any previously loaded debug symbols.

Note

Special care must be taken when placing this command into script functions (see *Avoiding Script Function Recursions* on page 197).

Prototype

```
int File.Load(const char* sFilePath, U64 Address);
```

Argument	Meaning
sFilePath	Path to a program or data file (see <i>File Path Arguments</i> on page 221).
Address	Memory address to download the data contents to. In case the address is provided by the file itself, 0 can be specified.

Return Value

-1: error
0: success

GUI Access

None

7.9.1.8 File.NewProject

Creates a new project (see *File Menu* on page 44).

Prototype

```
int File.NewProject();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → File → New → New Project (Ctrl+N)

7.9.1.9 File.NewProjectWizard

Opens the Project Wizard (see *Project Wizard* on page 34).

Prototype

```
int File.NewProjectWizard();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → File → New → New Project Wizard (Ctrl+Alt+N)

7.9.1.10 File.Open

Opens a file (see *File Menu* on page 44). When a program file is opened and the debug session is running, the program is automatically downloaded to target memory.

Note

Special care must be taken when placing this command into script functions (see *Avoiding Script Function Recursions* on page 197).

Prototype

```
int File.Open(const char* sFilePathOrName);
```

Argument	Meaning
sFilePathOr-Name	File path (or name) of a project-, source- or program-file (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Open (Ctrl+O)

7.9.1.11 File.OpenRecent

Reopens a recently opened program file.

Prototype

```
int File.OpenRecent(int Index);
```

Argument	Meaning
Index	Position of the file within the file menu's recent programs list, starting at index 0.

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Recent Programs

7.9.1.12 File.OpenProjectInEditor

Opens the project file within the source viewer.

Prototype

```
int File.OpenProjectInEditor();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Edit Project File

7.9.1.13 File.Reload

Reloads a document from disk. Note that unsaved document changes will be lost when this command is executed.

Prototype

```
int File.Reload(const char* sFilePathOrName);
```

Argument	Meaning
sFilePath	File path or name of a document currently open within the Source Viewer (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

Document Tab -> Reload from disk

7.9.1.14 File.SaveAll

Saves all modified files.

Prototype

```
int File.SaveAll();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Save all

7.9.1.15 File.SaveProjectAs

Saves the project file under a new file path to disk.

Prototype

```
int File.SaveProjectAs(const char* sFilePath);
```

Argument	Meaning
sFilePath	New project file path (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Save Project as (Ctrl+Shift+S)

7.9.1.16 File.Save

Saves an open document to disk.

Prototype

```
int File.Save(const char* sFilePathOrName);
```

Argument	Meaning
sFilePathOr-Name	File path (or name) of a document which is opened within the Source Viewer. When empty, the file path of the active document is used (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Save (Ctrl+S)

7.9.1.17 File.SaveAs

Saves an open document under a new file path to disk.

Prototype

```
int File.SaveAs(const char* sFilePathOrName, const char* sFilePathNew);
```

Argument	Meaning
sFilePathOr-Name	File path (or name) of a document which is opened within the Source Viewer. When empty, the file path of the active document is used (see <i>File Path Arguments</i> on page 221).
sFilePathNew	File path to save to.

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Save As...

7.9.1.18 File.SaveCopyAs

Saves a copy of an open document to disk.

Prototype

```
int File.SaveCopyAs(const char* sFilePathOrName, const char* sFilePathOut);
```

Argument	Meaning
sFilePathOr-Name	File path (or name) of a document which is opened within the Source Viewer. When empty, the file path of the active document is used (see <i>File Path Arguments</i> on page 221).
sFilePathOut	File path to save to.

Return Value

-1: error
0: success

GUI Access

Main Menu → File → Save Copy As...

7.9.1.19 File.SelectInExplorer

Selects a source file or a directory within the file explorer of the operating system.

Prototype

```
int File.SelectInExplorer(const char* sLocation);
```

Argument	Meaning
sLocation	File path (or name) of a source file or directory path (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

Source Viewer → File Tab → Select In Explorer (Ctrl+F2)

7.9.2 Find Actions

7.9.2.1 Find.Text

Shows the Quick Find Widget to locate a text pattern within the active document (see *Quick Find Widget* on page 91).

Prototype

```
int Find.Text();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Find → Find (Ctrl+F)

7.9.2.2 Find.TextInFiles

Opens the Find In Files Dialog (see *Find In Files Dialog* on page 69).

Prototype

```
int Find.TextInFiles();
```

Return Value

-1: error

0: success

GUI Access

Main Menu → Find → Find In Files (Ctrl+Shift+F)

7.9.2.3 Find.TextInTrace

Opens the Find In Trace Dialog (see *Find In Trace Dialog* on page 71).

Prototype

```
int Find.TextInTrace();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Find → Find In Trace (Ctrl+Shift+T)

7.9.2.4 Find.Function

Shows the Quick Find Widget to locate a program function (see *Quick Find Widget* on page 91).

Prototype

```
int Find.Function();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Find → Find Function (Ctrl+M)

7.9.2.5 Find.GlobalData

Shows the Quick Find Widget to locate a global variable (see *Quick Find Widget* on page 91).

Prototype

```
int Find.GlobalData();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Find → Find Global Data (Ctrl+J)

7.9.2.6 Find.SourceFile

Shows the Quick Find Widget to open a source file (see *Quick Find Widget* on page 91).

Prototype

```
int Find.SourceFile();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Find → Find Source File (Ctrl+K)

7.9.3 Tools Actions

7.9.3.1 Tools.JLinkSettings

Opens the J-Link Settings Dialog (see *J-Link Settings Dialog* on page 76).

Prototype

```
int Tools.JLinkSettings();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → J-Link Settings (Ctrl+Alt+J)

7.9.3.2 Tools.TraceSettings

Opens the Trace Settings Dialog (see *Trace Settings Dialog* on page 83).

Prototype

```
int Tools.TraceSettings();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → Trace Settings (Ctrl+Alt+T)

7.9.3.3 Tools.Preferences

Displays the User Preference Dialog (see *User Preference Dialog* on page 85).

Prototype

```
int Tools.Preferences();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → Preferences (Ctrl+Alt+P)

7.9.3.4 Tools.SysVars

Displays the System Variable Editor (see *System Variable Editor* on page 82).

Prototype

```
int Tools.SysVars();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → System Variables (Ctrl+Alt+V)

7.9.3.5 Tools.SemihostingSettings

Opens the Semihosting Settings Dialog (see *Semihosting Settings Dialog* on page 81).

Prototype

```
int Tools.SemihostingSettings();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → Semihosting Settings (Ctrl+Alt+H)

7.9.4 Edit Actions

7.9.4.1 Edit.Preference

Edits a user preference.

Prototype

```
int Edit.Preference(int ID, int Value);
```

Argument	Meaning
ID	User preference identifier (see <i>User Preference Identifiers</i> on page 272).
Value	User preference value. Certain user preferences are specified in a predefined format (see <i>Value Descriptors</i> on page 263).

Additional Information

User preferences can be alternatively edited using the User Preference Dialog (see *User Preference Dialog* on page 85).

Return Value

-1: error
0: success

GUI Access

None.

7.9.4.2 Edit.SysVar

Edits a system variable (see *System Variable Identifiers* on page 275).

Prototype

```
int Edit.SysVar(int ID, int Value);
```

Argument	Meaning
ID	System variable identifier (see <i>System Variable Identifiers</i> on page 275).
Value	System variable value. Certain system variable values are specified in a predefined format (see <i>Value Descriptors</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → System Variables (Ctrl+Alt+V)

7.9.4.3 Edit.Find

Searches a text pattern in the active document (see *Source Viewer* on page 155). Once executed, hotkey F3 can be used to locate the next occurrence.

Prototype

```
int Edit.Find(const char* sFindWhat);
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Find → Find (Ctrl+F)

7.9.4.4 Edit.Color

Edits an application color (see *Color Identifiers* on page 271).

Prototype

```
int Edit.Color(int ID, int Value);
```

Argument	Meaning
ID	Color identifier (see <i>Color Identifiers</i> on page 271).

Argument	Meaning
Value	Color descriptor (see <i>Color Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Main Menu → Edit → Preferences → Appearance

7.9.4.5 Edit.Font

Edits an application font (see *Font Identifiers* on page 270).

Prototype

```
int Edit.Font(int ID, const char* sFont);
```

Argument	Meaning
ID	Font identifier (see <i>Font Identifiers</i> on page 270).
sFont	Font descriptor (see <i>Font Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Main Menu → Edit → Preferences → Appearance

7.9.4.6 Edit.DisplayFormat

Edits an object's value display format.

Prototype

```
int Edit.DisplayFormat(const char* sObject, int Format);
```

Argument	Meaning
sObject	Name of a debug information window, program variable or register. Registers can be specified using the plain name (such as "MODER") or register window path name (such as "Peripherals.GPIO.GPIOA.MODER").
Format	Value Display Formats (see <i>Value Display Formats</i> on page 265).

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Display As

7.9.4.7 Edit.RefreshRate

Sets the refresh rate of a debug window or watched expression (see *Live Watches* on page 196).

Prototype

```
int Edit.RefreshRate (const char* sDest, int Frequency);
```

Argument	Meaning
sDest	Debug window name (e.g. "Registers 1") or C-Language expression (see <i>Working With Expressions</i> on page 204).
Frequency	Update frequency in Hz (see <i>Frequency Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Watched Data Window → Context Menu → Refresh Rate

7.9.4.8 Edit.MemZone

Assigns a memory zone to a watched expression (see *Live Watches* on page 196). Whenever an update of the expression's value is requested, the specified memory zone is accessed.

Prototype

```
int Edit.MemZone (const char* sExpression, const char* sMemZone);
```

Argument	Meaning
sExpression	C-Language expression (see <i>Working With Expressions</i> on page 204).
sMemZone	Memory zone name

Return Value

-1: error
0: success

GUI Access

Watched Data Window → Context Menu → Memory Zone

7.9.5 Export Actions

7.9.5.1 Export.CodeProfile

Exports the current code profile dataset.

Prototype

```
int Export.CodeProfile (const char* sFilePath, int Options, const char* sItem-  
sToExport);
```

Argument	Meaning
sFilePath	Destination text file (see <i>File Path Arguments</i> on page 221). An empty string prompts the export to be performed to a temporary file. The temporary file is opened within the Source Viewer and can be saved to disk later on.
Options	bitwise-OR combination of export option flags (see <i>Code Profile Export Options</i> on page 268). Use value 0 to specify default options.
sItemsToExport	A comma-separated list containing the names of the functions to export. The list may also contain source file (module) names, in which case all functions contained within the module are selected for export. An empty list (the default) selects all program functions for export.

Additional Information

Command Window.WaitForUpdateComplete can be employed to ensure that the Code Profile Window has processed all available sampling data before the export is performed.

Return Value

-1: error
0: success

GUI Access

Code Profile Window → Context Menu → Export...

7.9.5.2 Export.Disassembly

Exports program disassembly (see *Disassembly Export Dialog* on page 67).

Prototype

```
int Export.Disassembly(const char* sFilePath, const char* sFuncOrArange, U32 Flags);
```

Argument	Meaning
sFilePath	Output file path (see <i>File Path Arguments</i> on page 221). An empty string prompts the export to be performed to a temporary file. The temporary file is opened within the Source Viewer and can be saved to disk later on.
sFuncOrArange	A function name or an address range string of the format "<StartAddr>-<EndAddr>". When set, only the specified function or address range is exported. An empty string selects the whole program. This option requires the output format to be "CSV".
Flags	Bitwise-OR combination of export options (see <i>Disassembly Export Options</i> on page 269). This argument defaults to 0.

Return Value

-1: error
0: success

GUI Access

Disassembly Window → Context Menu → Export...

7.9.5.3 Export.DataGraphs

Exports all data graphs to a CSV file (see *Data Sampling Window* on page 113).

Prototype

```
int Export.DataGraphs(const char* sFilePath);
```

Argument	Meaning
sFilePath	Output file path (see <i>File Path Arguments</i> on page 221). An empty string prompts the export to be performed to a temporary file. The temporary file is opened within the Source Viewer and can be saved to disk later on.

Additional Information

Command Window.WaitForUpdateComplete can be employed to ensure that the Data Sampling Window has processed all available sampling data before the export is performed.

Return Value

-1: error
0: success

GUI Access

Data Sampling Window → Context Menu → Export...

7.9.5.4 Export.PowerGraphs

Exports power sampling data to a CSV file (see *Power Sampling Window* on page 144).

Prototype

```
int Export.PowerGraphs(const char* sFilePath);
```

Argument	Meaning
sFilePath	Output file path (see <i>File Path Arguments</i> on page 221). An empty string prompts the export to be performed to a temporary file. The temporary file is opened within the Source Viewer and can be saved to disk later on.

Additional Information

Command Window.WaitForUpdateComplete can be employed to ensure that the Power Sampling Window has processed all available sampling data before the export is performed.

Return Value

-1: error
0: success

GUI Access

Power Sampling Window → Context Menu → Export...

7.9.5.5 Export.Trace

Exports the contents of the Instruction Trace Window to a CSV file.

Prototype

```
int Export.Trace(const char* sFilePath, U64 InstCnt);
```

Argument	Meaning
sFilePath	Output file path (see <i>File Path Arguments</i> on page 221). An empty string prompts the export to be performed to a temporary file. The temporary file is opened within the Source Viewer and can be saved to disk later on.
MaxInstCnt	Maximum number of instructions to export. When not specified (0), this value defaults to the number of instructions currently loaded by the Instruction Trace Window. This value is limited by system variable VAR_TRACE_MAX_INST_CNT.

Additional Information

Command Window.WaitForUpdateComplete can be employed to ensure that the Instruction Trace Window has loaded and processed all available instructions before the export is performed.

Return Value

-1: error
0: success

GUI Access

Instruction Trace Window → Context Menu → Export...

7.9.6 Window Actions

7.9.6.1 Window.Show

Shows a window (see *Window Layout* on page 138).

Prototype

```
int Window.Show(const char* sWindow);
```

Argument	Meaning
sWindow	Name of the window (e.g. "Source Files"). See <i>View Menu</i> on page 45.

Return Value

-1: error
0: success

GUI Access

Main Menu → View → Window Name (Shift+Alt+Letter)

7.9.6.2 Window.Close

Closes a window (see *Window Layout* on page 138).

Prototype

```
int Window.Close(const char* sWindow);
```

Argument	Meaning
sWindow	Name of the window (e.g. "Source Files"). See <i>View Menu</i> on page 45.

Return Value

-1: error
0: success

GUI Access

Main Menu → Window → Close Window (Alt+X)

7.9.6.3 Window.CloseAll

Closes all windows (see *Window Layout* on page 138).

Prototype

```
int Window.CloseAll();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Window → Close All Window (Alt+Shift+X)

7.9.6.4 Window.SetDisplayFormat

Set's a window's value display format (see *Display Format* on page 57).

Prototype

```
int Window.SetDisplayFormat(const char* sWindow, int Format);
```

Argument	Meaning
sWindow	Name of the window (e.g. "Source Files"). See <i>View Menu</i> on page 45.
Format	Value display format (see <i>Value Display Formats</i> on page 265).

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Display All As (Alt+Number)

7.9.6.5 Window.ShowFullScreen

Activates or deactivates main window full screen mode.

Prototype

```
int Window.ShowFullScreen(int On);
```

Return Value

-1: error
0: success

GUI Access

Main Menu → View → Enter/Exit Full Screen (Alt+Shift+Return)

7.9.6.6 Window.Add

Adds a symbol to a debug window (see *Debug Information Windows* on page 94).

Prototype

```
int Window.Add(const char* sWindow, const char* sSymbol);
```

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Add (Alt+Plus)

7.9.6.7 Window.Insert

Inserts a symbol into a table window (see *Debug Information Windows* on page 94).

Prototype

```
int Window.Insert(const char* sWindow, const char* sSymbol, int TableRow);
```

Argument	Meaning
sWindow	Window name, as displayed within the window title.
sSymbol	Function name, variable or expression.
TableRow	insertion position.

Return Value

-1: error
0: success

GUI Access

None

7.9.6.8 Window.Insert

Inserts a symbol into a debug window (see *Debug Information Windows* on page 94).

Prototype

```
int Window.Insert (const char* sWindow, const char* sSymbol, int Row);
```

Argument	Meaning
sWindow	Name of the window (e.g. "Source Files"). See <i>View Menu</i> on page 45.

Argument	Meaning
sSymbol	Name of the symbol to insert.
Row	Insert symbol at this position. When empty, append the symbol.

Return Value

-1: error
0: success

GUI Access

None

7.9.6.9 Window.Remove

Removes a symbol from a debug window (see *Debug Information Windows* on page 94).

Prototype

```
int Window.Remove(const char* sWindow, const char* sSymbol);
```

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Remove (Del)

7.9.6.10 Window.Clear

Clears a window.

Prototype

```
int Edit.TerminalSettings();
```

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Clear (Alt+Del)

7.9.6.11 Window.ExpandAll

Expands all expandable window items.

Prototype

```
int Window.ExpandAll();
```

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Expand All (Alt+Plus)

7.9.6.12 Window.Export

Exports the contents of a debug window to file.

Prototype

```
int Window.Export(const char* sWindow, const char* sFilePath);
```

Argument	Meaning
sWindow	Window name, as displayed within the window title.
sFilePath	Output file path (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Export

7.9.6.13 Window.CollapseAll

Collapses all collapsible window items.

Prototype

```
int Window.CollapseAll();
```

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Collapse All (Alt+Minus)

7.9.6.14 Window.WaitForUpdateComplete

Waits until all debug information windows have finished updating following a change of the program execution point.

Prototype

```
int Window.WaitForUpdateComplete(U32 MaxTimeMillis);
```

Argument	Meaning
MaxTimeMillis	Maximum time to wait in milliseconds.

Return Value

-1: error
0: success

GUI Access

None

7.9.7 Toolbar Actions

7.9.7.1 Toolbar.Show

Displays a toolbar (see *Showing and Hiding Toolbars* on page 49).

Prototype

```
int Toolbar.Show(const char* sToolbar);
```

Return Value

-1: error
0: success

GUI Access

Main Menu → View → Toolbars → Toolbar Name

7.9.7.2 Toolbar.Close

Hides a toolbar (see *Showing and Hiding Toolbars* on page 49).

Prototype

```
int Toolbar.Show(const char* sToolbar);
```

Return Value

-1: error
0: success

GUI Access

Main Menu → View → Toolbars → Toolbar Name

7.9.7.3 Toolbar.AddCustomButton

Adds a button to the Custom Toolbar. The parameters specify the text to be displayed on the button and the function call that is to be executed each time the button is pressed.

Upon creation of the button it is not checked, whether the respective function call works or not. This becomes visible only after the button is pressed.

Each invocation of this function creates a new button. If a button with the same text already exists, the result will be two buttons with the same text. It is not possible to alter the command or text of a button after it was created.

Prototype

```
int Toolbar.AddCustomButton(const char* sButtonName, const char* sCommand);
```

Return Value

-1: error
0: success

GUI Access

None

7.9.7.4 Toolbar.RemoveCustomButton

Removes a button from the Custom Toolbar. The parameter specifies the button to be removed, numbering starts at 1.

Prototype

```
int Toolbar.RemoveCustomButton(int Number);
```

Return Value

-1: error
0: success

GUI Access

None

7.9.7.5 Toolbar.EnableCustomButton

Enables a button in the Custom Toolbar. The parameter specifies the button to be enabled, numbering starts at 1.

Upon creation a button is always enabled. It may be disabled using the command `Toolbar.DisableCustomButton`.

Prototype

```
int Toolbar.EnableCustomButton(int Number);
```

Return Value

-1: error
0: success

GUI Access

None

7.9.7.6 Toolbar.DisableCustomButton

Disables a button in the Custom Toolbar. The parameter specifies the button to be disabled, numbering starts at 1.

Upon creation a button is always enabled. It may be disabled using this command. Once it is disabled, it may be re-enabled using `Toolbar.EnableCustomButton`.

Prototype

```
int Toolbar.DisableCustomButton(int Number);
```

Return Value

-1: error
0: success

GUI Access

None

7.9.7.7 Toolbar.PressButton

Performs the same action as if clicking onto a button in a toolbar with the mouse. The parameters specify the name of the toolbar and the number of the button to be pressed, numbering starts at 1.

Pressing a disabled button does not have any effect.

Prototype

```
int Toolbar.PressButton(const char* sToolbar, int Number);
```

Return Value

-1: error
0: success

GUI Access

Click on the respective button

7.9.8 Utility Actions

7.9.8.1 Util.Error

Shows an error message box and optionally stops the debug session.

Prototype

```
int Util.Error(const char* sError, int StopDebug);
```

Argument	Meaning
sError	Error message
StopDebug	1: Ask the user if the debug session is to be stopped. 2: Stop the debug session. Otherwise, simply show the error message. Optional.

Return Value

-1: error
0: success

GUI Access

None

7.9.8.2 Util.Log

Prints a message to the Console Window (see *Console Window* on page 110).

Prototype

```
int Util.Log(const char* sMessage);
```

Return Value

-1: error
0: success

GUI Access

None

Note

This command supports escaped quotes.

7.9.8.3 Util.LogHex

Appends an integer value to a text message and prints the result to the Console Window (see *Console Window* on page 110).

Prototype

```
int Util.LogHex(const char* sMessage, unsigned int IntValue);
```

Return Value

-1: error
0: success

GUI Access

None

Note

This command supports escaped quotes.

7.9.8.4 Util.Sleep

Pauses the current operation for a given amount of time.

Prototype

```
int Util.Sleep(int milliseconds);
```

Return Value

-1: error
0: success

GUI Access

None

7.9.9 Script Actions

7.9.9.1 Script.Exec

Executes a project file script function. The command currently only supports script functions with void parameter or with up to seven arguments of integer type.

Prototype

```
int Script.Exec(const char* sFuncName, __int64 Para1, __int64 Para2,...);
```

Return Value

Return value of the executed function (-1 if execution failed).

GUI Access

None

7.9.9.2 Script.DefineConst

Defines a constant integer value to be used within the project file script.

Prototype

```
int Script.DefineConst(const char* sName, const char* sExpression);
```

Argument	Meaning
sName	Name of the constant.
sExpression	Symbol expression that evaluates to a numeric value of size ≤ 8 bytes (see <i>Working With Expressions</i> on page 204). The symbol expression cannot contain local variables.

Return Value

-1: error
0: success

GUI Access

None

7.9.10 Show Actions**7.9.10.1 Show.Memory**

Displays a memory location within the Memory Window (see *Memory Window* on page 134).

Prototype

```
int Show.Memory(U64 Address);
```

Return Value

-1: error
0: success

GUI Access

Memory Window → Context Menu → Go To (Ctrl+G)

7.9.10.2 Show.MemoryMap

Displays a symbol within the Memory Usage Window (see *Memory Usage Window* on page 138).

Prototype

```
int Show.MemoryMap(const char* sSymbol);
```

Return Value

-1: error
0: success

GUI Access

Source Viewer → Context Menu → Show in Memory Map (Ctrl+B)

7.9.10.3 Show.Source

Displays the source code location of a variable, function or machine instruction within the Source Viewer (see *Source Viewer* on page 155).

Prototype

```
int Show.Source(const char* sLocation);
```

Argument	Meaning
sLocation	Variable name: displays the source code declaration of a variable. Function name: displays the source code implementation of a function. Memory address: displays the source line affiliated with an instruction. Source location: displays a particular source location (see <i>Source Code Location Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Symbol Windows → Context Menu → Show Source (Ctrl+U)

7.9.10.4 Show.ValueSource

Displays the symbol pointed to within the Source Viewer.

Prototype

```
int Show.ValueSource(const char* sExpression);
```

Argument	Meaning
sExpression	Variable name: name of a function or data pointer. Memory address: memory location of a function or data pointer. Register name: register location of a function or data pointer.

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Show Value in Source

7.9.10.5 Show.ValueDisassembly

Displays the symbol pointed to within the Disassembly Window.

Prototype

```
int Show.ValueDisassembly(const char* sExpression);
```

Argument	Meaning
sExpression	Variable name: name of a function or data pointer. Memory address: memory location of a function or data pointer. Register name: register location of a function or data pointer.

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Show Value in Disassembly

7.9.10.6 Show.ValueData

Displays the symbol pointed to within the Memory Window.

Prototype

```
int Show.ValueData(const char* sExpression);
```

Argument	Meaning
sExpression	Variable name: name of a function or data pointer. Memory address: memory location of a function or data pointer. Register name: register location of a function or data pointer.

Return Value

-1: error
0: success

GUI Access

Window → Context Menu → Show Value in Data

7.9.10.7 Show.Data

Displays the data location of a global or local program variable within the Registers Window (see *Registers Window* on page 146) or the Memory Window (see *Memory Window* on page 134).

Prototype

```
int Show.Data(const char* sVariable);
```

Return Value

-1: error
0: success

GUI Access

Symbol Windows → Context Menu → Show Data (Ctrl+T)

7.9.10.8 Show.Disassembly

Displays the assembly code of a function or source code statement within the Disassembly Window (see *Disassembly Window* on page 116).

Prototype

```
int Show.Disassembly(const char* sLocation);
```

Argument	Meaning
sLocation	Function name: displays the disassembly of a function. Memory address: displays the disassembly at a memory location. Source location: displays the disassembly of a source statement (see <i>Source Code Location Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Symbol Windows → Context Menu → Show Disassembly (Ctrl+D)

7.9.10.9 Show.Definition

Displays the source code definition location of a symbol within the Source Viewer (see *Source Viewer* on page 155).

Prototype

```
int Show.Definition(const char* sSymbol);
```

Return Value

-1: error
0: success

GUI Access

Source Viewer → Context Menu → Show Definition (F12)

7.9.10.10 Show.Declaration

Displays the source code declaration location of a symbol within the Source Viewer (see *Source Viewer* on page 155).

Prototype

```
int Show.Declaration(const char* sSymbol);
```

Return Value

-1: error
0: success

GUI Access

Source Viewer → Context Menu → Show Declaration (Shift+F12)

7.9.10.11 Show.CallGraph

Displays the call graph of a function.

Prototype

```
int Show.CallGraph (const char* sFuncName);
```

Return Value

-1: error
0: success

GUI Access

→ Source Viewer → Context Menu → Show Call Graph (Ctrl+H)

7.9.10.12 Show.InstTrace

Displays a position in the history (stack) of executed machine instructions.

Prototype

```
int Show.InstTrace (int StackPos);
```

Argument	Meaning
StackPos	Position 1 = most recently executed machine instruction.

Return Value

-1: error
0: success

GUI Access

Instruction Trace Window → Context Menu → Go To

7.9.10.13 Show.Line

Displays a text line in the active document.

Prototype

```
int Show.Line(unsigned int Line);
```

Return Value

-1: error
0: success

GUI Access

Source Viewer → Context Menu → Go To Line (Ctrl+L)

7.9.10.14 Show.PC

Displays the program's execution point within the Disassembly Window (see *Disassembly Window* on page 116).

Prototype

```
int Show.PC();
```

Return Value

-1: error
0: success

GUI Access

Disassembly Window → Context Menu → Go To PC (Ctrl+P)

7.9.10.15 Show.PCLine

Displays the program's execution point within the Source Viewer (see *Source Viewer* on page 155).

Prototype

```
int Show.PCLine();
```

Return Value

-1: error
0: success

GUI Access

Source Viewer → Context Menu → Go To PC (Ctrl+P)

7.9.10.16 Show.NextResult

Displays the next search result.

Prototype

```
int Show.NextResult();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.10.17 Show.PrevResult

Displays the previous search result.

Prototype

```
int Show.PrevResult();
```

Return Value

-1: error
0: success

GUI Access

None.

7.9.11 Snapshot Actions

7.9.11.1 Snapshot.SaveReg

Saves a register or register group to a snapshot (see *Register Groups* on page 147).

Prototype

```
int Snapshot.SaveReg(const char* sReg);
```

Argument	Meaning
sReg	Plain register (group) name (such as "MODER") or register window path name (such as "Peripherals.GPIO.GPIOA.MODER"). The latter variant is obligatory when the input is a vendor-specific peripheral register (group).

Return Value

-1: error
0: success

GUI Access

None

7.9.11.2 Snapshot.SaveU32

Saves a 32 bit integer value to a snapshot.

Prototype

```
int Snapshot.SaveU32(U64 Addr, U32 Value);
```

Argument	Meaning
Addr	Address of the integer value to store. The integer is written to this target memory location when the snapshot is loaded.
Value	32-bit integer value. The value is converted to target endianness before download.

Return Value

-1: error
0: success

GUI Access

None

7.9.11.3 Snapshot.ReadReg

Reads a register value from a snapshot (see *Register Groups* on page 147).

Prototype

```
int Snapshot.ReadReg(const char* sReg);
```

Argument	Meaning
sReg	Plain register name (such as "MODER") or register window path name (such as "Peripherals.GPIO.GPIOA.MODER"). The latter variant is obligatory when the input is a vendor-specific peripheral register. When a register group was stored to the snapshot, each group register can be accessed individually.

Return Value

-1 when register (or containing group) is not stored in snapshot, otherwise register value.

GUI Access

None

7.9.11.4 Snapshot.ReadU32

Reads a 32 bit value from a snapshot.

Prototype

```
int Snapshot.ReadU32(U64 Addr);
```

Argument	Meaning
Addr	Target memory address.

Return Value

-1 when the snapshot does not contain integer data for the given address, otherwise data.

GUI Access

None

7.9.11.5 Snapshot.LoadReg

Reads a register (group) from a snapshot and writes it to target (see *Register Groups* on page 147).

Prototype

```
int Snapshot.LoadReg(const char* sReg);
```

Argument	Meaning
sReg	Plain register (group) name (such as "MODER") or register window path name (such as "Peripherals.GPIO.GPIOA.MODER"). The latter variant is obligatory when the input is a vendor-specific peripheral register (group). When a register group was stored to the snapshot, each group register can be accessed individually. When a register group is loaded to the target, registers are written piece-wise.

Return Value

-1: error, i.e. register (or containing group) not stored in snapshot
 0: success

GUI Access

None

7.9.11.6 Snapshot.LoadU32

Reads a 32 bit value from a snapshot and writes it to target.

Prototype

```
int Snapshot.LoadU32(U64 Addr);
```

Argument	Meaning
Addr	Target memory address.

Return Value

-1: error, i.e. snapshot does not contain integer data at the given address or value could not be downloaded.
 0: success

GUI Access

None

7.9.12 Debug Actions**7.9.12.1 Debug.Start**

Starts the debug session (see *Starting the Debug Session* on page 183). The startup routine can be reprogrammed (see *TargetConnect* on page 228).

Prototype

```
int Debug.Start();
```

Return Value

-1: error
 0: success

GUI Access

Main Menu → Debug → Start Debugging (F5)

7.9.12.2 Debug.Stop

Closes the debug session (see *Closing the Debug Session* on page 222).

Prototype

```
int Debug.Stop();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Debug → Stop Debugging (Shift+F5)

7.9.12.3 Debug.Disconnect

Disconnects the debugger from the target.

Prototype

```
int Debug.Disconnect();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.12.4 Debug.Connect

Establishes a connection to the target and starts the debug session in the default way. A reprogramming of the startup procedure via script function "Target-Connect" is ignored.

Prototype

```
int Debug.Connect();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.12.5 Debug.SetConnectMode

Sets the connection mode (see *Connection Mode* on page 183).

Prototype

```
int Debug.SetConnectMode(int Mode);
```

Argument	Meaning
Mode	Connection mode (see <i>Connection Modes</i> on page 266).

Return Value

-1: error
0: success

GUI Access

None

7.9.12.6 Debug.Continue

Resumes program execution (see *Resume* on page 190).

Prototype

```
int Debug.Continue();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Debug → Continue (F5)

7.9.12.7 Debug.Halt

Halts program execution (see *Halt* on page 190).

Prototype

```
int Debug.Halt();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Debug → Halt (Ctrl+F5)

7.9.12.8 Debug.Reset

Resets the target and the debuggee (see *Reset* on page 189). The reset operation can be customized via the scripting interface (see [TargetReset](#)).

Prototype

```
int Debug.Reset();
```

Argument	Meaning
Mode	Reset mode (see <i>Reset Modes</i> on page 266).

Return Value

-1: error
0: success

GUI Access

Main Menu → Debug → Reset (F4)

7.9.12.9 Debug.SetResetMode

Sets the reset mode. The reset mode determines how the program is reset (see *Reset Mode* on page 189).

Prototype

```
int Debug.SetResetMode(int Mode);
```

Return Value

-1: error
0: success

GUI Access

None

7.9.12.10 Debug.StepInto

Steps into the current subroutine (see *Step* on page 189).

Prototype

```
int Debug.StepInto();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Debug → Step Into (F11)

7.9.12.11 Debug.StepOver

Steps over the current subroutine (see *Step* on page 189).

Prototype

```
int Debug.StepOver();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Debug → Step Over (F12)

7.9.12.12 Debug.StepOut

Steps out of the current subroutine. (see *Step* on page 189).

Prototype

```
int Debug.StepOut();
```

Return Value

-1: error

0: success

GUI Access

Main Menu → Debug → StepOut (Shift+F11)

7.9.12.13 Debug.SetNextPC

Sets the execution point to a particular machine instruction (see *Execution Point* on page 194).

Prototype

```
int Debug.SetNextPC(U64 Address);
```

Return Value

-1: error
0: success

GUI Access

Disassembly Window → Context Menu → Set Next PC (Shift+F10)

7.9.12.14 Debug.SetNextStatement

Sets the execution point to a particular source code line (see *Execution Point* on page 194).

Prototype

```
int Debug.SetNextStatement(const char* sStatement);
```

Argument	Meaning
sStatement	Function name: displays the first source line of a function. Source location: displays a particular source location (see <i>Source Code Location Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Source Viewer → Context Menu → Set Next Statement (Shift+F10)

7.9.12.15 Debug.RunTo

Advances the program execution point to a particular source code line, function or instruction address (see *Execution Point* on page 194).

Prototype

```
int Debug.RunTo(const char* sLocation);
```

Argument	Meaning
sStatement	Function name: advances program execution to the first source line of a function. Memory address: advances program execution to a particular instruction address.

Argument	Meaning
	Source location: advances program execution to a particular source code line (see <i>Source Code Location Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Code Window → Context Menu → Run To Cursor (Ctrl+F10)

7.9.12.16 Debug.Download

Downloads the debuggee to the target (see *Program Files* on page 182). The download operation can be reprogrammed (see [TargetDownload](#)).

Prototype

```
int Debug.Download();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.12.17 Debug.ReadIntoInstCache

Reads a machine code block into Ozone's instruction cache (see *Setting Up The Instruction Cache* on page 208). The preferred way to employ this command is to call it from project script function [OnStartupComplete](#).

Prototype

```
int Debug.ReadIntoInstCache(U64 Address, U32 Size);
```

Argument	Meaning
Address	Start address of the target memory block to be read into the instruction cache.
Size	Byte size of the target memory block to be read into the instruction cache.

Return Value

-1: error
0: success

GUI Access

None

7.9.12.18 Debug.IsHalted

Queries the program state.

Prototype

```
int Debug.IsHalted();
```

Return Value

- 0: Program is running
- 1: Program is halted

GUI Access

None

7.9.12.19 Debug.LoadSnapshot

Loads a debug snapshot (see *Snapshot Dialog* on page 78).

Prototype

```
int Debug.LoadSnapshot(const char* sFilePath);
```

Argument	Meaning
sFilePath	Snapshot file path (*.jsnap, see <i>File Path Arguments</i> on page 221).

Return Value

- 1: error
- 0: success

GUI Access

Debug → Load Snapshot

7.9.12.20 Debug.SaveSnapshot

Saves a debug snapshot (see *Snapshot Dialog* on page 78).

Prototype

```
int Debug.SaveSnapshot(const char* sFilePath, unsigned int Flags);
```

Argument	Meaning
sFilePath	Snapshot file path (*.jsnap, see <i>File Path Arguments</i> on page 221).
Flags	Bitwise-OR combination of individual debug snapshot settings (see <i>Snapshot Save Flags</i> on page 269. This argument defaults to 0.

Return Value

- 1: error
- 0: success

GUI Access

Debug → Save Snapshot

7.9.13 Help Actions

7.9.13.1 Help.About

Shows the About Dialog.

Prototype

```
int Help.About();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Help → About

7.9.13.2 Help.UserGuide

Opens the user guide within the default PDF viewer.

Prototype

```
int Help.UserGuide();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Help → User Guide (F1)

7.9.13.3 Help.ReleaseNotes

Opens the release notes within the web browser.

Prototype

```
int Help.ReleaseNotes();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Help → Release Notes

7.9.13.4 Help.LicenseManager

Opens the license manager.

Prototype

```
int Help.LicenseManager();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Help → License Manager

7.9.13.5 Help.Commands

Prints the command help to the Console Window (see *Command Help* on page 111)

Prototype

```
int Help.Commands();
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Help → Commands (Shift+F1)

7.9.14 Process Actions

7.9.14.1 Process.Exec

Spawns a process and executes an external application.

The exit code of the application is conveyed in the return value of this command.

Text output of the application to `stdout` and `stderr` is captured and displayed in the console window in dedicated colors. The colors can be changed, see *Message Colors*.

A timeout in milliseconds is specified. If the application does not terminate before the timeout is reached, the application will be killed. In that case the command will return the value -1.

Setting the timeout to 0 will launch the application in a fire-and-forget way; in that case the command will return 0 and output on `stdout` and `stderr` will neither be captured nor be displayed in the console window.

Only a single external application can be executed at the same time. The next application cannot be started before the previous application has terminated. This limit does not apply to applications started in the fire-and-forget way.

Quotes in the command line and/or argument list need to be escaped by a preceding backslash (i.e. `\"`).

Prototype

```
int Process.Exec(const char* sCmd, const char* sArgs, int Timeout);
```

Argument	Meaning
sCmd	The command line for launching the process to be spawned.
sArgs	The argument list to be passed to the process, formatted in a single string.
Timeout	The timeout in milliseconds.

Return Value

-1: error: Timeout occurred or process could not be spawned.
otherwise: the exit code of the Process

Note

An application returning the exit code -1 cannot be distinguished from an application that could not be started or that was killed due to the timeout since in all those cases the command returns the value -1.

Note

This command supports escaped quotes.

7.9.15 Project Actions

7.9.15.1 Project.SetDevice

Specifies the target device (see *J-Link Settings Dialog* on page 76).

Prototype

```
int Project.SetDevice(const char* sDeviceName);
```

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → J-Link Settings (Ctrl+Alt+J)

7.9.15.2 Project.SetFlashLoader

Specifies the flash loader configuration, i.e. the flash loader(s) to be used for one or multiple flash bank(s).

The configuration string consists of a list of tupels, each tuple specifying the base address of a flash bank and the name of the flash loader to be used for that flash bank, both separated by an equal sign character ("="). Multiple tupels are separated by a semicolon (;). Specifying one tuple is mandatory, specifying more than one tuple is optional.

```
"<BankAddress> = <LoaderName> [ ; <BankAddress2> = <LoaderName2> ] .. [ ; <BankAddress_n> = <LoaderName_n> ]"
```

For flash banks which are not listed in the configuration string, J-Link will use the respective bank's default flash loader.

This function shall be invoked only from within `OnProjectLoad()` in the Ozone project script.

Prototype

```
int Project.SetFlashLoader(const char* sConfig);
```

Argument	Meaning
sConfig	The flash loader configuration string

Return Value

-1: error

0: success

GUI Access

None

7.9.15.3 Project.SetHostIF

Specifies the host interface (see *Host Interfaces* on page 265).

Prototype

```
int Project.SetHostIF(const char* sHostIF, const char* sHostID);
```

Argument	Meaning
sHostIF	Host interface (see <i>Host Interfaces</i> on page 265).
sHostID	Host identifier (USB serial number or IP address).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → J-Link Settings (Ctrl+Alt+J)

7.9.15.4 Project.SetTargetIF

Specifies the target interface (see *Target Interfaces* on page 265).

Prototype

```
int Project.SetTargetIF(const char* sTargetIF);
```

Argument	Meaning
sTargetIF	Target interface (see <i>Target Interfaces</i> on page 265).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → J-Link Settings (Ctrl+Alt+J)

7.9.15.5 Project.SetTIFSpeed

Specifies the target interface speed (see *J-Link Settings Dialog* on page 76).

Prototype

```
int Project.SetTIFSpeed(const char* sFrequency);
```

Argument	Meaning
sFrequency	Frequency Descriptor (see <i>Frequency Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → J-Link Settings (Ctrl+Alt+J)

7.9.15.6 Project.SetJTAGConfig

Configures the JTAG target interface scan chain parameters.

Prototype

```
int Project.SetJTAGConfig(int DRPre, int IRPre);
```

Argument	Meaning
DRPre	Position of the target in the JTAG scan chain. 0 is closest to TDO.
IRPre	Sums of IR-Lens of devices closer to TDO. IRLen of ARM devices is 4.

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → J-Link Settings (Ctrl+Alt+J)

7.9.15.7 Project.SetBPTType

Sets the permitted breakpoint implementation type, i.e. restricts breakpoints to be implemented in the way specified by the command argument.

Prototype

```
int Project.SetBPTType(int Type);
```

Argument	Meaning
Type	Breakpoint Implementation Types (see <i>Breakpoint Implementation Types</i> on page 266).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → System Variables (Ctrl+Alt+V)

7.9.15.8 Project.SetCorePlugin

Sets the file path of the plugin that provides target support (see *Target Support Plugins* on page 27). Applying this setting causes the debugger's automatic plugin selection to be overridden.

Prototype

```
int Project.SetCorePlugin(const char* sFilePath);
```

Argument	Meaning
sFilePath	Plugin file path or name. Valid plugin file extensions are .dll on Windows, .so on linux and .dylib on macOS. The file path may be specified case-insensitively on all platforms. For further details, see <i>File Path Arguments</i> on page 221.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.9 Project.SetDisassemblyPlugin

Sets the file path of the plugin that provides disassembly support for custom instructions (see *Disassembly Plugin* on page 231).

Prototype

```
int Project.SetDisassemblyPlugin(const char* sFilePath);
```

Argument	Meaning
sFilePath	File path to a JavaScript plugin file. The file path may be specified case-insensitively on all platforms. For further details, see <i>File Path Arguments</i> on page 221.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.10 Project.SetOSPlugin

Specifies the file path or name of the plugin that adds RTOS awareness to the debugger.

Prototype

```
int Project.SetOSPlugin(const char* sFilePath);
```

Argument	Meaning
sFilePath	Plugin file path or name. The valid plugin file extension is .js on all platforms. The file path may be specified case-insensitively on all platforms. The file extension may be omitted. For further details, see <i>File Path Arguments</i> on page 221.

The provided plugins may be found in the section *Available RTOS Plugins* on page 151.

7.9.15.11 Project.SetSmartViewPlugin

Specifies the file path or name of a SmartView plugin to the debugger.

Prototype

```
int Project.SetSmartViewPlugin(const char* sFilePath);
```

Argument	Meaning
sFilePath	Plugin file path or name.

Additional Description

Multiple SmartView scripts may be loaded at the same time. In that case Project.SetSmartViewPlugin needs be invoked once for each script.

A programming guide for SmartView plugins is provided by section *SmartView Awareness Plugin* on page .

7.9.15.12 Project.SetRTT

Enables or disables the Real-Time Transfer interface (see *Real-Time Transfer* on page 199).

Prototype

```
int Project.SetRTT(int OnOff);
```

Return Value

-1: error
0: success

GUI Access

Terminal Window → Context Menu → Capture RTT

7.9.15.13 Project.AddRTTSearchRange

Specifies a memory range to be considered during RTT initialization, specifically RTT control block discovery (see *Real-Time Transfer* on page 199).

The RTT control block is a data structure located at the address of global program variable `_SEGGER_RTT`. When the program file provides a debug symbol for variable `_SEGGER_RTT`, no discovery is necessary. RTT discovery refers to the process of searching target RAM for a byte pattern specific to the control block in order to locate it. To speed up this process, command Project.AddRTTSearchRange is provided.

For further details, refer to the [J-Link User Guide](#) .

Prototype

```
int Project.AddRTTSearchRange(U32 StartAddr, U32 Size);
```

Argument	Meaning
StartAddr – Size	Address range to be considered in the RTT buffer localization routine.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.14 Project.SetTraceSource

Selects the trace source to be used.

Prototype

```
int Project.SetTraceSource(const char* sTraceSrc);
```

Argument	Meaning
sTraceSrc	Display name of the trace source to be used (see <i>Trace Sources</i> on page 268).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → Trace Settings (Ctrl+Alt+T)

7.9.15.15 Project.ConfigSemihosting

Configures the Semihosting interface (see *Semihosting* on page 200).

Note

Ozone automatically enables semihosting on the first CPU halt after debug session start. When not required, semihosting can be explicitly disabled by setting ModeBP, ModeBKPT and ModeSVC to No.

Prototype

```
int Project.ConfigSemihosting(const char* sConfig);
```

Argument	Meaning
sConfig	Settings string of the format <code>setting1=value1;setting2=value2,...</code> . The available settings are listed below. The default value of each setting is highlighted.

AllowOpenRead

Sets the permission for semihosting operation SysOpen when file flag `READ` is set.

Value	Description
0 (Ask)	A popup dialog is shown which asks the user if the operation should be performed.
1 (Yes)	The operation is always allowed
2 (No)	The operation is never allowed

AllowOpenWrite

Sets the permission for semihosting operation SysOpen when file flag `WRITE` is set.

Value	Description
0 (Ask)	A popup dialog is shown which asks the user if the operation should be performed.
1 (Yes)	The operation is always allowed
2 (No)	The operation is never allowed

AllowRename

Sets the permission for semihosting operation SysRename.

Value	Description
0 (Ask)	A popup dialog is shown which asks the user if the operation should be performed.
1 (Yes)	The operation is always allowed
2 (No)	The operation is never allowed

AllowRemove

Sets the permission for semihosting operation SysRemove.

Value	Description
0 (Ask)	A popup dialog is shown which asks the user if the operation should be performed.
1 (Yes)	The operation is always allowed
2 (No)	The operation is never allowed

ModeSVC

Enables or disables semihosting via the SVC instruction.

Value	Description
0 (Yes, ask on non-semihosting SVC)	Semihosting via SVC enabled. A vector catch will be set on the SVC exception at debug session start. A popup dialog will be shown each time the program stops on the vector catch, but not due to a semihosting request (i.e. when a non-semihosting software interrupt was triggered).
1 (Yes)	Semihosting via SVC enabled. A vector catch will be set on the SVC exception at debug session start.
2 (No)	Semihosting via SVC disabled. No vector catch will be set for semihosting on debug session start.

ModeBKPT

Enables or disables semihosting via the BKPT instruction (ARM) / EBREAK instruction (RISC-V).

Value	Description
0 (Yes)	Semihosting via BKPT/EBREAK enabled.
1 (No and continue)	Semihosting via BKPT/EBREAK disabled. When the debuggee halts on a BKPT/EBREAK semihosting trap instruction, it is automatically resumed by Ozone.
2 (No and halt)	Semihosting via BKPT/EBREAK disabled. The debuggee halts on BKPT/EBREAK semihosting trap instructions.

ModeBP

Enables or disables semihosting via the generic trap instruction.

Value	Description
0 (Yes)	Semihosting on breakpoint enabled. Ozone will set a hidden breakpoint on address <code>BPAddress</code> on debug session start in order to serve semihosting requests.
1 (No)	Semihosting on breakpoint disabled. No hidden breakpoint is set on debug session start.

InputViaTerminal

Sets the user input mode.

Value	Description
0 (No)	User input is obtained via a popup dialog that is shown each time Ozone receives a semihosting input request from the debuggee.
1 (Yes)	User input is obtained via the terminal prompt, which gets highlighted and focused each time Ozone receives a semihosting input request from the debuggee.

ExitMode

Specifies Ozone's reaction on a `SYS_EXIT` semihosting operation.

Value	Description
0 (No)	The debug session is ended. <code>Debug.Stop()</code> is executed.
1 (Yes)	The target is halted. <code>Debug.Halt()</code> is executed.

Vector

When semihosting is in its default configuration state and enabled, Ozone will set a vector catch on the SVC instruction at address `0x8` in order to catch semihosting requests. This default behavior can reduce the run-time performance of clients which make extensive use of software interrupts. In order to alleviate this problem, Ozone provides semihosting configuration setting `Vector`.

Setting `Vector` instructs Ozone to set a hidden breakpoint on arbitrary address `Vector` within the SVC handler instead of setting a vector catch on SVC. The breakpointed instruction then acts as the semihosting SVC trap instead of the vector catch. This way, developers get the chance to evaluate the SWI opcode within the SVC handler on the target side. The handler code is expected to execute the trap instruction only when the SWI opcode matches a semihosting SWI opcode. When this option is employed, developers have to make sure that the semihosting opcode and argument pointer registers R0, R1 and R2 are not modified within SVC handler code up to the point where the trap instruction is executed.

SVCNumberThumb

Edits the SWI number definition of the 16-bit thumb SVC semihosting trap instruction. The default value for this setting is `0xAB`. The valid range for this value is `0-0xFF`. As an example, when SVC semihosting requests are to be performed via instruction `SVC #0x10`, then this setting should be set to value `0x10`.

SVCNumberARM

Edits the SWI number definition of the 32-bit ARM SVC semihosting trap instruction. The default value for this setting is `0x123456`. The valid range for this value is `0-0xFFFFFFFF`. As an example, when SVC semihosting requests are to be performed via instruction `SVC #0x1234`, then this setting should be set to value `0x1234`.

BKPTNumber

Edits the software breakpoint number definition of the BKPT semihosting trap instruction. The default value for this setting is 0xAB. The valid range for this value is 0-0xFF. As an example, when SVC semihosting requests are to be performed via instruction BKPT #0x10, then this setting should be set to value 0x10.

BPAddress

Edits the address of the generic semihosting trap instruction. The default value for this setting is the base address of function `SEGGER_SEMIHOST_DebugHalt`. The valid value range for this setting is the address range of function `SEGGER_SEMIHOST_DebugHalt`. Depending on setting `ModeBP`, Ozone will or will not set a hidden breakpoint on the configured address in order to catch generic semihosting requests by the debuggee.

TargetCmdLine

Sets the command line text that Ozone is to transmit to the debuggee when it receives semihosting request `SysGetCmdLine`. This is the only setting that can not be edited via the settings dialog.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.16 Project.SetTracePortWidth

Specifies the number of trace pins (data lines) comprising the target's trace port. This setting is only relevant when the selected trace source is "Trace Pins" / ETM (see *Project.SetTraceSource* on page 344).

Prototype

```
int Project.SetTracePortWidth(int PortWidth);
```

Argument	Meaning
PortWidth	Number of trace data lines provided by the target. Possible values are 1, 2 or 4.

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → Trace Settings (Ctrl+Alt+T)

7.9.15.17 Project.SetTraceTiming

This command adjusts the trace pin sampling delays. The delays may be necessary in case the target hardware does not provide sufficient setup and hold times for the trace pins. In such cases, delaying TCLK can compensate this and make tracing possibly anyhow. This setting is only relevant when the selected trace source is "Trace Pins" / ETM (see *Project.SetTraceSource* on page 344).

Prototype

```
int Project.SetTraceTiming(int d1, int d2, int d3, int d4);
```

Argument	Meaning
dn	Trace data pin n sampling delay in picoseconds. Only the first parameters are relevant when your hardware has less than 4 trace pins.

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → Trace Settings (Ctrl+Alt+T)

7.9.15.18 Project.ConfigSWO

Configures the Serial Wire Output (SWO) interface (see *SWO* on page 199). This setting is only relevant when the selected trace source is SWO (see *Project.SetTraceSource* on page 344).

Prototype

```
int Project.ConfigSWO(const char* sSWOFreq, char* sCPUFreq);
```

Argument	Meaning
sSWOFreq	Specifies the data transmission speed on the SWO interface (see <i>Frequency Descriptor</i> on page 263).
sCPUFreq	Specifies the target's processor frequency (see <i>Frequency Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → Trace Settings (Ctrl+Alt+T)

7.9.15.19 Project.SetMemZoneRunning

Specifies the default memory zone that is accessed when the program is running. The debugger uses this memory zone for any memory access that has not been explicitly assigned to a particular memory zone.

Prototype

```
int Project.SetMemZoneRunning(const char* sMemoryZone);
```

Argument	Meaning
sMemoryZone	Name of the default memory zone

Return Value

-1: error

0: success

GUI Access

Main Menu → Tools → System Variables (Ctrl+Alt+V)

7.9.15.20 Project.AddSvdFile

Adds a register set description file to be loaded by the Registers Window (see *SVD Files* on page 146).

Prototype

```
int Project.AddSvdFile(const char* sFilePath);
```

Argument	Meaning
sFilePath	Path to a CMSIS-SVD file. Both .svd and .xml file extensions are supported. For further details, see <i>File Path Arguments</i> on page 221.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.21 Project.AddFileAlias

Adds a file path alias (see *File Path Resolution Sequence* on page 206).

Prototype

```
int Project.AddFileAlias(const char* sFilePath, const char* sAliasPath);
```

Argument	Meaning
sFilePath	Original file path as it appears within the program file or elsewhere.
sAliasPath	Replacement for the original file path.

Return Value

-1: error
0: success

GUI Access

Source Files Window → Context Menu → Locate File (Space)

7.9.15.22 Project.AddRootPath

Adds a source file root path. The root path helps the debugger resolve relative file path arguments (see *File Path Resolution Sequence* on page 206). Typically a project will have a single source file root path.

Prototype

```
int Project.SetRootPath(const char* sRootPath);
```

Argument	Meaning
sRootPath	Fully qualified path of a file system directory.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.23 Project.AddPathSubstitute

Replaces a substring within unresolved source file path arguments (see *File Path Resolution Sequence* on page 206).

Prototype

```
int Project.AddPathSubstitute(const char* sSubStr, const char* sAlias);
```

Argument	Meaning
sSubStr	Substring (directory name) within original file paths.
sAlias	Replacement for the given substring.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.24 Project.AddSearchPath

Adds a directory to the list of search directories. Search directories help the debugger resolve invalid file path arguments (see *File Path Resolution Sequence* on page 206).

Prototype

```
int Project.AddSearchPath(const char* sSearchPath);
```

Argument	Meaning
sSearchPath	Fully qualified path of a file system directory.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.25 Project.SetJLinkScript

Specifies the J-Link script file that is to be executed at the moment the debug session is started. Refer to the [J-Link User Guide](#) for an overview on J-Link script files.

Prototype

```
int Project.SetJLinkScript(const char* sFilePath);
```

Argument	Meaning
sFilePath	Path to a J-Link script file (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

None

7.9.15.26 Project.SetJLinkLogFile

Specifies the text file that receives J-Link logging output or, in case of a GDB server being connected, the communication between GDB server and Ozone's GDB client.

Prototype

```
int Project.SetJLinkLogFile(const char* sFilePath);
```

Argument	Meaning
sFilePath	Path to a text file (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

None

7.9.15.27 Project.RelocateSymbols

Relocates one or multiple symbols. The command must be executed before the ELF program file is opened. It is currently not supported to execute the command at program run-time. Furthermore, relocating symbols outside of their containing ELF data section address range is currently not supported. When an ELF data section lies completely within a relocated address range, it is relocated together with all containing symbols.

Prototype

```
int Project.RelocateSymbols(const char* sSymbols, int Offset);
```

Argument	Meaning
sSymbols	Specifies the symbols to be relocated. The wildcard character "*" selects all symbols. A symbol name specifies a single symbol. A section name such as ".text" specifies a particular ELF data section.

Argument	Meaning
Offset	The offset that is added to the base addresses of all specified symbols.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.28 Project.SetConsoleLogFile

Sets the text file to which Console Window messages are logged.

Prototype

```
int Project.SetConsoleLogFile(const char* sFilePath);
```

Argument	Meaning
sFilePath	Log file path (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

None

7.9.15.29 Project.SetTerminalLogFile

Sets the text file to which Terminal Window messages are logged.

Prototype

```
int Project.SetTerminalLogFile(const char* sFilePath);
```

Argument	Meaning
sFilePath	Log file path (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

None

7.9.15.30 Project.ConfigDisassembly

Configures the disassembler.

Prototype

```
int Project.ConfigDisassembly(unsigned int Flags);
```

Argument	Meaning
Flags	Bitwise-OR combination of individual flags. Each flag specifies a disassembler option. Refer to <i>Disassembler Option Flags</i> on page 267 for the list of supported options.

Return Value

-1: error
0: success

GUI Access

None

7.9.15.31 Project.DisableSessionSave

Selects session information that is not to be saved to the user file.

Prototype

```
int Project.DisableSessionSave(unsigned int Flags);
```

Argument	Meaning
Flags	Bitwise-OR combination of individual flags. Each flag specifies a session information that is not to be saved to (and restored from) the user file. Refer to <i>Session Save Flags</i> on page 269 for the list of supported flags.

Return Value

-1: error
0: success

GUI Access

None

7.9.16 Code Profile Actions

7.9.16.1 Profile.Exclude

Filters program entities from the code profile (load) statistic. The code profile statistic is re-evaluated as if the filtered items had never belonged to the program.

Prototype

```
int Profile.Exclude (const char* sFilter);
```

Argument	Meaning
sFilter	Specifies the items to be filtered. All items that exactly match the filter string are moved to the filtered set. Wildcard (*) characters can be placed at the front or end of the filter string to perform partial match filtering.

Return Value

-1: error
0: success

GUI Access

Code Profile Window → Context Menu → Exclude...

7.9.16.2 Profile.Include

Re-adds filtered items to the code profile load statistic.

Prototype

```
int Profile.Include (const char* sFilter);
```

Argument	Meaning
sFilter	Specifies the items to be unfiltered. All items that exactly match the filter string are removed from the filtered set. Wildcard (*) characters can be placed at the front or end of the filter string to perform partial match unfiltering.

Return Value

-1: error
0: success

GUI Access

Code Profile Window → Context Menu → Include...

7.9.16.3 Profile.Reset

Clears code profile data and resets all execution counters.

Prototype

```
int Profile.Reset ();
```

Return Value

-1: error
0: success

GUI Access

Code Profile Window → Context Menu → Reset Execution Counters

7.9.16.4 Coverage.Exclude

Filters program entities from the code coverage statistic. The code coverage statistic is re-evaluated as if the filtered items had never belonged to the program.

Prototype

```
int Coverage.Exclude (const char* sFilter);
```

Argument	Meaning
sFilter	Specifies the items to be filtered. All items that exactly match the filter string are moved to the filtered set. Wildcard (*) characters can

Argument	Meaning
	be placed at the front or end of the filter string to perform partial match filtering.

Return Value

-1: error
0: success

GUI Access

Code Profile Window → Context Menu → Exclude...

7.9.16.5 Coverage.Include

Re-adds filtered items to the code coverage statistic.

Prototype

```
int Coverage.Include (const char* sFilter);
```

Argument	Meaning
sFilter	Specifies the items to be unfiltered. All items that exactly match the filter string are removed from the filtered set. Wildcard (*) characters can be placed at the front or end of the filter string to perform partial match unfiltering.

Return Value

-1: error
0: success

GUI Access

Code Profile Window → Context Menu → Include...

7.9.16.6 Coverage.ExcludeNOPs

Excludes instructions without operation (alignment instructions) from the code coverage statistics.

Prototype

```
int Coverage.ExcludeNOPs ();
```

Return Value

-1: error
0: success

GUI Access

Code Profile Window → Context Menu → Exclude All Trailing NOPs...

7.9.17 Register Actions

7.9.17.1 Register.Addr

Returns the memory location of a target register.

Prototype

```
int Register.Addr(const char* sReg);
```

Argument	Meaning
sReg	Plain register name (such as "MODER") or register window path name (such as "Peripherals.GPIO.GPIOA.MODER"). The latter variant is obligatory when the input is a vendor-specific peripheral register.

Return Value

Target memory address or -1 on invalid input (e.g. when not a memory-mapped register).

GUI Access

None

7.9.18 Target Actions

7.9.18.1 Target.EraseChip

Erases all of the target's FLASH memory by writing all data bytes to 0xFF.

Prototype

```
int Target.EraseChip();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.18.2 Target.SetReg

Writes a target register (see *Register Groups* on page 147).

Prototype

```
int Target.SetReg(const char* sReg, unsigned int Value);
```

Argument	Meaning
sReg	Plain register name (such as "MODER") or register window path name (such as "Peripherals.GPIO.GPIOA.MODER"). The latter variant is obligatory when the input is a vendor-specific peripheral register. System registers can also be specified using an architecture-specific notation, as described in <i>System Register Descriptor</i> on page 264.
Value	Register value to write.

Return Value

-1: error

0: success

GUI Access

Register Window → Register

7.9.18.3 Target.GetReg

Reads a target register (see *Register Groups* on page 147).

Prototype

```
U32 Target.GetReg(const char* sReg);
```

Argument	Meaning
sReg	Plain register name (such as "MODER") or register window path name (such as "Peripherals.GPIO.GPIOA.MODER"). The latter variant is obligatory when the input is a vendor-specific peripheral register. System registers can also be specified using an architecture-specific notation, as described in <i>System Register Descriptor</i> on page 264.

Return Value

-1: error
register value: success

GUI Access

Register Window → Register

7.9.18.4 Target.WriteU32

Writes a word to target memory (see *Target Memory* on page 195).

Prototype

```
int Target.WriteU32(U64 Address, U32 Value);
```

Return Value

-1: error
0: success

GUI Access

Memory Window

7.9.18.5 Target.WriteU16

Writes a half word to target memory (see *Target Memory* on page 195).

Prototype

```
int Target.WriteU16(U64 Address, U16 Value);
```

Return Value

-1: error
0: success

GUI Access

Memory Window

7.9.18.6 Target.WriteU8

Writes a byte to target memory (see *Target Memory* on page 195).

Prototype

```
int Target.WriteU8(U64 Address, U8 Value);
```

Return Value

-1: error
0: success

GUI Access

Memory Window

7.9.18.7 Target.ReadU32

Reads a word from target memory (see *Target Memory* on page 195).

Prototype

```
U32 Target.ReadU32(U64 Address);
```

Return Value

-1: error
Memory value: success

GUI Access

Memory Window

7.9.18.8 Target.ReadU16

Reads a half word from target memory (see *Target Memory* on page 195).

Prototype

```
U16 Target.ReadU16(U64 Address);
```

Return Value

-1: error
Memory value: success

GUI Access

Memory Window

7.9.18.9 Target.ReadU8

Reads a byte from target memory (see *Target Memory* on page 195).

Prototype

```
U32 Target.ReadU8(U64 Address);
```

Return Value

-1: error
Memory value: success

GUI Access

Memory Window

7.9.18.10 Target.SetAccessWidth

Specifies the default access width to be used when accessing target memory (see *Target.SetAccessWidth* on page 359).

Prototype

```
int Target.SetAccessWidth(U32 AccessWidth);
```

Argument	Meaning
AccessWidth	Memory access width (See <i>Memory Access Widths</i> on page 265).

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → System Variables (Ctrl+Alt+V)

7.9.18.11 Target.FillMemory

Fills a block of target memory with a particular value (see *Target.FillMemory* on page 359).

Prototype

```
int Target.FillMemory(U64 Address, U32 Size, U8 FillValue);
```

Argument	Meaning
Address	Start address of the memory block to fill.
Size	Size of the memory block to fill.
FillValue	Value to fill the memory block with.

Return Value

-1: error
0: success

GUI Access

Memory Window → Context Menu → Fill (Ctrl+I)

7.9.18.12 Target.FillMemoryEx

Fills a block of target memory with a particular value (see *Target.FillMemoryEx* on page 359) and a particular word width.

The value can be seen as a pattern which is used to fill the memory and the word width specifies the period of the pattern. Supported values for width are 1, 2, 3, 4 and 8 bytes, so the memory area can be filled byte-wise, half-word-wise, word-wise and double-word-

wise. In addition, frame buffers containing information organized in 3 bytes, can directly be filled, as well.

Prototype

```
int Target.FillMemoryEx(U64 Address, U32 Size, U8 Width, U64 FillValue);
```

Argument	Meaning
Address	Start address of the memory block to fill.
Size	Size of the memory block to fill (in bytes).
Width	The word width.
FillValue	Value to fill the memory block with.

Return Value

-1: error
0: success

GUI Access

Memory Window → Context Menu → Fill (Ctrl+I)

7.9.18.13 Target.SaveMemory

Saves a block of target memory to a binary data file (see *Target.SaveMemory* on page 360).

Prototype

```
int Target.SaveMemory(const char* sFilePath, U64 Address, U32 Size);
```

Argument	Meaning
sFilePath	Output binary data file (*.bin, see <i>File Path Arguments</i> on page 221).
Address	Start address of the memory block to save to the destination file.
Size	Size of the memory block to save to the destination file.

Return Value

-1: error
0: success

GUI Access

Memory Window → Context Menu → Save

7.9.18.14 Target.LoadMemory

Downloads the contents of a binary data file to target memory (see *Download Behavior Comparison* on page 197).

Prototype

```
int Target.LoadMemory(const char* sFilePath, U64 Address);
```

Argument	Meaning
sFilePath	Input binary data file (*.bin, see <i>File Path Arguments</i> on page 221).
Address	Download address.

Return Value

-1: error
0: success

GUI Access

Memory Window → Context Menu → Load

7.9.18.15 Target.SetEndianness

Sets the data endianness mode of the target.

Prototype

```
int Target.SetEndianness(int BigEndian);
```

Argument	Meaning
BigEndian	When 0, little endian is selected. Otherwise, big endian is selected.

Return Value

-1: error
0: success

GUI Access

Main Menu → Tools → J-Link-Settings → Target Device (Ctrl+Alt+J)

7.9.18.16 Target.LoadMemoryMap

Loads a memory map from an Embedded Studio memory map file. The loaded memory map is applied to the Memory Usage Window (see *Supplying Memory Segment Information* on page 139).

Prototype

```
int Target.LoadMemoryMap(const char* sFilePath);
```

Argument	Meaning
sFilePath	Path to a memory map file. Currently, the only supported file format is SEGGER Embedded Studio. For further details, see <i>File Path Arguments</i> on page 221.

Return Value

-1: error
0: success

GUI Access

Memory Usage Window → Context Menu → Edit Regions

break

7.9.18.17 Target.AddMemorySegment

Adds a segment to the memory map displayed by the Memory Usage Window (see *Supplying Memory Segment Information* on page 139).

Prototype

```
int Target.AddMemorySegment(const char* sName, U64 Addr, U32 Size);
```

Argument	Meaning
sName	Segment name.
Addr	Segment base address.
Size	Segment byte size.

Return Value

-1: error
0: success

GUI Access

Memory Usage Window → Context Menu → Edit Regions

7.9.18.18 Target.PowerOn

Enables or disables target power supply via the debug probe.

Prototype

```
int Target.PowerOn(int On);
```

Argument	Meaning
On	When 1, the target is powered via the debug probe. When 0, target power via J-Link/J-Trace is switched off.

Return Value

-1: error
0: success

GUI Access

Main Menu → Edit → System Variables (Ctrl+Alt+V)

7.9.19 Timeline Actions

7.9.19.1 Timeline.Reset

Resets all panes of the timeline window, i.e. the session's trace and sampling data (see *Timeline Window* on page 164).

Prototype

```
int Timeline.Reset();
```

Return Value

-1: error
0: success

GUI Access

Timeline Window

7.9.20 J-Link Actions

→ Reset Data (Ctrl+Alt+V)

7.9.20.1 Exec.Connect

Establishes a connection to the target (see [DebugStart](#)).

Prototype

```
int Exec.Connect();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.20.2 Exec.Reset

Performs a hardware reset of the target (see [DebugStart](#)).

Prototype

```
int Exec.Reset();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.20.3 Exec.Download

Downloads the contents of a program file to target memory (see *Download Behavior Comparison* on page 197 and *Supported Program File Types* on page 182).

Prototype

```
int Exec.Download(const char* sFilePath);
```

Argument	Meaning
sFilePath	Path to a program or data file (see <i>File Path Arguments</i> on page 221).

Return Value

-1: error
0: success

GUI Access

None

7.9.20.4 Exec.Command

Executes a J-Link command.

Prototype

```
int Exec.Command(const char* sCommand);
```

Argument	Meaning
sCommand	J-Link command to execute (refer to the J-Link User Guide for on overview on the available commands).

Return Value

-1: error
0: success

GUI Access

None

7.9.20.5 Exec.AddCommandOnOpen

Schedules a J-Link command to be executed immediately before or after opening the J-Link connection (i.e. `JLink_Open()` is invoked). This function shall be invoked only from within `OnProjectLoad()` in the Ozone project script.

Prototype

```
int Exec.AddCommandOnOpen(const char* sCommand, int BeforeNotAfterOpen);
```

Argument	Meaning
sCommand	J-Link command to execute (refer to the J-Link User Guide for on overview on the available commands).
BeforeNotAfterOpen	Indicates whether the command is to be executed immediately before (1) or after (0) opening a J-Link connection.

Return Value

-1: error
0: success

GUI Access

None

7.9.21 OS Actions

7.9.21.1 OS.AddContextSwitchSymbol

Specifies a function or program instruction that performs a task switch when executed. This command can be used to enable a consistent output within the Timeline Window even when no RTOS Awareness Plugin was loaded (see *Timeline Window* on page 164).

Prototype

```
int OS.AddContextSwitchSymbol(const char* sSymbol);
```

Argument	Meaning
sSymbol	Function name, assembly label or instruction address.

Return Value

-1: error
0: success

GUI Access

None

7.9.22 Breakpoint Actions

7.9.22.1 Break.Set

Sets an instruction breakpoint (see *Instruction Breakpoints* on page 191).

Prototype

```
int Break.Set(U64 Address);
```

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set / Clear (Ctrl+Alt+B)

7.9.22.2 Break.SetEx

Sets an instruction breakpoint of a particular implementation type (see *Instruction Breakpoints* on page 191).

Prototype

```
int Break.SetEx(U64 Address, int Type);
```

Argument	Meaning
Address	Instruction address.
Type	Breakpoint Implementation Types (see <i>Breakpoint Implementation Types</i> on page 266).

Return Value

-1: error
0: success

GUI Access

None

7.9.22.3 Break.SetOnSrc

Sets a source breakpoint (see *Source Breakpoints* on page 191).

Prototype

```
int Break.SetOnSrc(const char* sLocation);
```

Argument	Meaning
sLocation	Name of a program function (e.g. "Reset_Handler") or source location (e.g. "main.c:100", see <i>Source Code Location Descriptor</i> on page 263).

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set / Clear (Ctrl+Alt+B)

7.9.22.4 Break.SetOnSrcEx

Sets a source breakpoint of a particular implementation type (see *Source Breakpoints* on page 191).

Prototype

```
int Break.SetOnSrc(const char* sLocation, int Type);
```

Argument	Meaning
sLocation	Name of a program function (e.g. "Reset_Handler") or source location (e.g. "main.c:100", see <i>Source Code Location Descriptor</i> on page 263).
Type	Breakpoint Implementation Types (see <i>Breakpoint Implementation Types</i> on page 266).

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set / Clear (Ctrl+Alt+B)

7.9.22.5 Break.SetType

Sets a breakpoint's permitted implementation type (see *Breakpoint Implementation Types* on page 266).

Prototype

```
int Break.SetType(const char* sLocation, int Type);
```

Argument	Meaning
sLocation	The breakpoint's source location (e.g. "main.c:100", see <i>Source Code Location Descriptor</i> on page 263) or instruction address.
Type	Breakpoint Implementation Types (see <i>Breakpoint Implementation Types</i> on page 266).

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Edit (F8)

7.9.22.6 Break.Clear

Clears an instruction breakpoint (see *Instruction Breakpoints* on page 191).

Prototype

```
int Break.Clear(U64 Address);
```

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set / Clear (Ctrl+Alt+B)

7.9.22.7 Break.ClearOnSrc

Clears a source breakpoint (see *Source Breakpoints* on page 191).

Prototype

```
int Break.ClearOnSrc(const char* sLocation);
```

Parameter Description

Refer to *Break.SetOnSrc* on page 365.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set / Clear (Ctrl+Alt+B)

7.9.22.8 Break.Enable

Enables an instruction breakpoint (see *Instruction Breakpoints* on page 191).

Prototype

```
int Break.Enable(U64 Address);
```

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Enable (Ctrl+F9)

7.9.22.9 Break.Disable

Disables an instruction breakpoint (see *Instruction Breakpoints* on page 191).

Prototype

```
int Break.Disable(U64 Address);
```

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Disable (Ctrl+F9)

7.9.22.10 Break.EnableOnSrc

Enables a source breakpoint (see *Source Breakpoints* on page 191).

Prototype

```
int Break.EnableOnSrc(const char* sLocation);
```

Parameter Description

Refer to *Break.SetOnSrc* on page 365.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Enable (Ctrl+F9)

7.9.22.11 Break.DisableOnSrc

Disables a source breakpoint (see *Source Breakpoints* on page 191).

Prototype

```
int Break.DisableOnSrc(const char* sLocation);
```

Parameter Description

Refer to *Break.SetOnSrc* on page 365.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Disable (Ctrl+F9)

7.9.22.12 Break.Edit

Edits a breakpoint's advanced properties.

Prototype

```
int Break.Edit(const char* sLocation, const char* sCondition, int DoTriggerOnChange, int SkipCount, const char* sTaskFilter, const char* sConsoleMsg, const char* sMsgBoxMsg);
```

Argument	Meaning
sLocation	The breakpoint's source location (e.g. "main.c:100", see <i>Source Code Location Descriptor</i> on page 263) or instruction address.
sCondition	Symbol expression that must evaluate to non-zero for the breakpoint to be triggered (see <i>Working With Expressions</i> on page 204).
DoTriggerOnChange	Indicates whether the condition is met when the expression value has changed since the last time it was evaluated (DoTriggerOnChange=1) or when it does not equal zero (DoTriggerOnChange=0).
SkipCount	Indicates how many times the breakpoint is skipped, i.e. how many times the program is resumed when the breakpoint is hit.
sTaskFilter	The name or ID of the RTOS task that triggers the breakpoint. When empty, all RTOS tasks trigger the breakpoint. The task filter is only operational when an RTOS plugin was specified using command <code>Project.SetOSPlugin</code> .
sConsoleMsg	Message printed to the Console Window when the breakpoint is triggered.
sMsgBoxMsg	Message displayed in a message box when the breakpoint is triggered.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Edit (F8)

7.9.22.13 Break.SetOnData

Sets a data breakpoint (see *Data Breakpoints* on page 193).

Prototype

```
int Break.SetOnData(U64 Address, U64 AddressMask, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Argument	Meaning
Address	Memory address that is monitored for IO (access) events.
AddressMask	Specifies which bits of the address are ignored when monitoring access events. By means of the address mask, a single data breakpoint can be set to monitor accesses to several individual memory addresses.
AccessType	Type of access that is monitored by the data breakpoint (see <i>Connection Modes</i> on page 266).
AccessSize	Access size condition required to trigger the data breakpoint. As an example, a data breakpoint with an access size of 4 bytes (word) will only be triggered when a word is written to one of the monitored memory locations. It will not be triggered when, say, a byte is written.

Argument	Meaning
MatchValue	Value condition required to trigger the data breakpoint. A data breakpoint will only be triggered when the match value is written to or read from one of the monitored memory addresses.
ValueMask	Indicates which bits of the match value are ignored when monitoring access events. A value mask of <code>0xFFFFFFFF</code> means that all bits are ignored, i.e. the value condition is disabled.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set (Ctrl+Alt+D)

7.9.22.14 Break.ClearOnData

Clears a data breakpoint (see *Data Breakpoints* on page 193).

Prototype

```
int Break.ClearOnData(U64 Address, U64 AddressMask, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnData* on page 369.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Clear (Ctrl+Alt+D)

7.9.22.15 Break.ClearAll

Clears all breakpoints (see *Data Breakpoints* on page 193).

Prototype

```
int Break.ClearAll();
```

Return Value

-1: error
0: success

GUI Access

Breakpoint Toolbar → Clear All Breakpoints

7.9.22.16 Break.ClearAllOnData

Clears all data breakpoints (see *Data Breakpoints* on page 193).

Prototype

```
int Break.ClearAllOnData();
```

Return Value

-1: error
0: success

GUI Access

Breakpoint Toolbar → Clear All Data Breakpoints

7.9.22.17 Break.EnableOnData

Enables a data breakpoint (see *Data Breakpoints* on page 193).

Prototype

```
int Break.EnableOnData(U64 Address, U64 AddressMask, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnData* on page 369.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Enable (Ctrl+F9)

7.9.22.18 Break.DisableOnData

Disables a data breakpoint (see *Data Breakpoints* on page 193).

Prototype

```
int Break.DisableOnData(U64 Address, U64 AddressMask, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnData* on page 369.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Disable (Ctrl+F9)

7.9.22.19 Break.EditOnData

Edits a data breakpoint (see *Data Breakpoints* on page 193).

Prototype

```
int Break.EditOnData(U64 Address, U64 AddressMask, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnData* on page 369.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Edit (F8)

7.9.22.20 Break.SetOnSymbol

Sets a data breakpoint on a symbol (see *Data Breakpoints* on page 193).

Prototype

```
int Break.SetOnSymbol(const char* sSymbolName, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Argument	Meaning
sSymbolName	Name of the symbol that is monitored by the data breakpoint.
AccessType	Type of access that is monitored by the data breakpoint (see <i>Access Types</i> on page 266).
AccessSize	Memory access size required to trigger the data breakpoint (see <i>Memory Access Widths</i> on page 265).
MatchValue	Value condition required to trigger the data breakpoint. A data breakpoint will only be triggered when the match value is written to or read from one of the monitored memory addresses.
ValueMask	Indicates which bits of the match value are ignored when monitoring access events. A value mask of 0xFFFFFFFF means that all bits are ignored, i.e. the value condition is disabled.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set (Ctrl+Alt+D)

7.9.22.21 Break.OnChange

Sets a data breakpoint on a symbol that triggers when the symbol value changes (see *Data Breakpoints* on page 193).

Prototype

```
int Break.OnChange(const char* sSymbolName);
```

Argument	Meaning
sSymbolName	Name of the symbol that is monitored by the data breakpoint.

Return Value

-1: error
0: success

GUI Access

Source Viewer → Context Menu → Break On Change

7.9.22.22 Break.ClearOnSymbol

Clears a data breakpoint on a symbol (see *Data Breakpoints* on page 193).

Prototype

```
int Break.ClearOnSymbol(const char* sSymbolName, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnSymbol* on page 372.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Clear (Ctrl+Alt+D)

7.9.22.23 Break.EnableOnSymbol

Enables a data breakpoint on a symbol (see *Data Breakpoints* on page 193).

Prototype

```
int Break.EnableOnSymbol(const char* sSymbolName, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnSymbol* on page 372.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Enable (Ctrl+F9)

7.9.22.24 Break.DisableOnSymbol

Disables a data breakpoint on a symbol (see *Data Breakpoints* on page 193).

Prototype

```
int Break.DisableOnSymbol(const char* sSymbolName, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnSymbol* on page 372.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Disable (Ctrl+F9)

7.9.22.25 Break.EditOnSymbol

Edits a data breakpoint on a symbol (see *Data Breakpoints* on page 193).

Prototype

```
int Break.EditOnSymbol (const char* sSymbolName, U8 AccessType, U8 AccessSize, U32 MatchValue, U32 ValueMask);
```

Parameter Description

Refer to *Break.SetOnSymbol* on page 372.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Edit (F8)

7.9.22.26 Break.SetCommand

Assigns a script function to a breakpoint that is executed when the breakpoint is hit.

Note

Due to hardware limitations, break point callback functions are not supported for data break points.

Prototype

```
int Break.SetCommand (const char* sLocation, const char* sFuncName);
```

Argument	Meaning
sLocation	The breakpoint's source location (e.g. "main.c:100", see <i>Source Code Location Descriptor</i> on page 263) or instruction address.
sFuncName	Name of the script function to callback when the breakpoint is hit.

Return Value

-1: error

0: success

GUI Access

Breakpoint Window → Context Menu → Edit (F8)

7.9.22.27 Break.SetCmdOnAddr

Assigns a script function to a breakpoint that is executed when the breakpoint is hit.

Note

Due to hardware limitations, break point callback functions are not supported for data break points.

Prototype

```
int Break.SetCmdOnAddr (U64 Address, const char* sFuncName);
```

Argument	Meaning
Address	Instruction address.
sFuncName	Name of the script function to callback when the breakpoint is hit.

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Edit (F8)

7.9.22.28 Break.SetVectorCatch

Edits the vector catch state.

Prototype

```
int Break.SetVectorCatch (U32 IndexMask);
```

Argument	Meaning
IndexMask	A bitmask where bit <n> corresponds to the vector catch at table row <n> of the <i>Breakpoints/Tracepoints Window</i> on page 95. Vector catch <n> is activated by setting bit <n>. A bitmask of 0 clears all vector catches.

Return Value

-1: error
0: success

GUI Access

Breakpoints/Tracepoints Window → Context Menu → Vector Catches

7.9.23 ELF Actions

7.9.23.1 Elf.GetBaseAddr

Returns the program file's download address. This is the lowest memory address written to during program download.

ARM specific:

The address returned by Elf.GetBaseAddr may or may not correspond to the base address of the program's vector table. The addresses in particular do not coincide when a code section is linked before the vector table, such as a bootloader.

Prototype

```
int Elf.GetBaseAddr();
```

Return Value

-1:	error
Base address:	success

GUI Access

None

7.9.23.2 Elf.GetFileClass

Returns the ELF file class of the program file. The ELF file class may be 32-bit or 64-bit.

Prototype

```
int Elf.GetFileClass();
```

Return Value

0:	none / invalid
1:	32-bit architecture
2:	64-bit architecture

GUI Access

None

7.9.23.3 Elf.GetEntryPointPC

Returns the initial PC of program execution.

Prototype

```
int Elf.GetEntryPointPC();
```

Return Value

Initial PC of program execution (-1 on error)

GUI Access

None

7.9.23.4 Elf.GetEntryFuncPC

Returns the base address of the program's entry (or main) function.

Prototype

```
int Elf.GetEntryFuncPC();
```

Return Value

PC of the program entry function (-1 on error)

GUI Access

None

7.9.23.5 Elf.GetExprValue

Evaluates a symbol expression.

Prototype

```
int Elf.GetExprValue(const char* sExpression);
```

Return Value

-1: error
Expression value: success

GUI Access

Watched Data Window → Context Menu → Add (Alt+Shift+Plus)

7.9.23.6 Elf.GetEndianness

Returns the program file's data encoding scheme.

Prototype

```
int Elf.GetEndianness(const char* sExpression);
```

Return Value

-1: indeterminable
0: Little Endian
1: Big Endian

GUI Access

None

7.9.23.7 Elf.SetConfig

Configures the ELF parser.

Prototype

```
int Elf.SetConfig(U32 ConfigFlags);
```

Argument	Meaning
ConfigFlags	Bitwise-or combination of ELF parser configuration flags (see <i>ELF Config Flags</i> on page 270).

Return Value

0: OK

-1: Error

GUI Access

None

7.9.23.8 Elf.PrintSectionInfo

Prints ELF file section information to the Console Window.

Prototype

```
int Elf.PrintSectionInfo(int SortCol);
```

Argument	Meaning
SortCol	0: sort output by name, 1: sort output by address

Return Value

0: OK
-1: Error

GUI Access

None

7.9.24 Trace Actions

Actions performing trace related operations.

7.9.24.1 Trace.SetPoint

Sets a tracepoint

Prototype

```
int Trace.SetPoint(int Op, const char* sLocation);
```

Argument	Meaning
Op	Operation to be performed when the tracepoint is hit (see <i>Tracepoint Operation Types</i> on page 268).
sLocation	Location of the tracepoint as displayed within the Breakpoints/Tracepoints Window (see <i>Breakpoints/Tracepoints Window</i> on page 95)).

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Set Tracepoint (Ctrl+Alt+E)

7.9.24.2 Trace.ClearPoint

Clears a tracepoint.

Prototype

```
int Trace.SetPoint(const char* sLocation);
```

Argument	Meaning
sLocation	Location of the tracepoint as displayed within the Breakpoints/Tracepoints Window (see <i>Breakpoints/Tracepoints Window</i> on page 95)).

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Clear (Ctrl+Alt+E)

7.9.24.3 Trace.EnablePoint

Enables a tracepoint.

Prototype

```
int Trace.EnablePoint(const char* sLocation);
```

Argument	Meaning
sLocation	Location of the tracepoint as displayed within the Breakpoints/Tracepoints Window (see <i>Breakpoints/Tracepoints Window</i> on page 95)).

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Enable (Ctrl+F9)

7.9.24.4 Trace.DisablePoint

Disables a tracepoint.

Prototype

```
int Trace.DisablePoint(const char* sLocation);
```

Argument	Meaning
sLocation	Location of the tracepoint as displayed within the Breakpoints/Tracepoints Window (see <i>Breakpoints/Tracepoints Window</i> on page 95)).

Return Value

-1: error
0: success

GUI Access

Breakpoint Window → Context Menu → Disable (Ctrl+F9)

7.9.24.5 Trace.ClearAllPoints

Clears all tracepoints.

Prototype

```
int Trace.ClearAllPoints();
```

Return Value

-1: error
0: success

GUI Access

Breakpoint Toolbar → Clear All Tracepoints

7.9.24.6 Trace.Reset

Resets Ozone's trace data buffer and the contents of the Instruction Trace Window.

Prototype

```
int Trace.Reset();
```

Return Value

-1: error
0: success

GUI Access

None

7.9.25 Watch Actions

7.9.25.1 Watch.Add

Adds an expression to the Watched Data Window (see *Watched Data Window* on page 175).

Prototype

```
int Watch.Add(const char* sExpression);
```

Return Value

-1: error
0: success

GUI Access

Watched Data Window → Context Menu → Add (Alt+Shift+Plus)

7.9.25.2 Watch.Insert

Inserts an expression into the Watched Data Window (see *Watched Data Window* on page 175).

Prototype

```
int Watch.Insert(const char* sExpression, int Row);
```

Argument	Meaning
sExpression	Ozone symbol expression. See <i>Working With Expressions</i> on page 204.
Row	Insert expression at this table row. When empty, append the expression.

Return Value

-1: error
0: success

GUI Access

None

7.9.25.3 Watch.Remove

Removes an expression from the Watched Data Window (see *Watched Data Window* on page 175).

Prototype

```
int Watch.Remove(const char* sExpression);
```

Return Value

-1: error
0: success

GUI Access

Watched Data Window → Context Menu → Remove (Del)

7.9.25.4 Watch.Quick

Shows an expression within the Quick Watch Dialog (see *Quick Watch Dialog* on page 93).

Prototype

```
int Watch.Quick(const char* sExpression);
```

Argument	Meaning
sExpression	Ozone symbol expression. See <i>Working With Expressions</i> on page 204.

Return Value

-1: error
0: success

GUI Access

Shift+F9